END-TO-END MEMORY NETWORKS: RE-IMPLEMENTATION AND SEMI-SUPERVISED EXTENSION ON BABI

Oloruntobi Paul Olutola, Sukhjot Kaur, Julie Dornat
Department of Computer Science
Paris Saclay University
Orsay, 91400, France

ABSTRACT

End-to-End Memory Networks (MemN2N) (Sukhbaatar et al., 2015) address multi-step reasoning over text by pairing an external memory with a differentiable, multi-hop attention mechanism trained entirely through backpropagation. We present a PyTorch re-implementation evaluated on the bAbI question-answering benchmark (Weston et al., 2015), together with a semi-supervised extension that uses supporting-fact annotations to guide hop-1 attention during training. Our MemN2N implementation reproduces the core architectural choices of the original paper which includes position encoding, temporal encoding, adjacent weight tying, and the Linear Start schedule, and achieves a mean test error of 10.8% across all 20 tasks, compared to 6.6% in the paper, with most of the gap concentrated on a small set of hard tasks. We then ask a more targeted question: when the model fails on a task, is it because it is attending to the wrong sentences? We test this by supervising the attention on Tasks 16, 17, and 19 at four labeling rates (0–50%). The answer depends entirely on the task: supervision nearly solves Task 16, slightly hurts Task 17, and does nothing for Task 19, revealing that these three tasks fail for fundamentally different reasons.

1 INTRODUCTION

Answering questions from a short passage of text sounds simple, but it requires a surprising variety of skills: retrieving the right sentence, chaining multiple facts together, understanding spatial relations, following logical rules. Standard recurrent networks (Hochreiter & Schmidhuber, 1997) compress the entire story into a single hidden vector before answering, which works well when the answer is close to the end of the story but tends to break down when relevant facts are spread far apart, or when several facts need to be combined.

Memory Networks (Weston et al., 2014) and their end-to-end variant MemN2N (Sukhbaatar et al., 2015) address this by keeping the story in an explicit memory and reading it multiple times. At each *hop*, the model computes a soft attention over all stored sentences, produces a weighted summary, and updates its internal query state. After K hops the updated state is used to predict the answer. The whole process is differentiable and trained end-to-end with backpropagation, making it a compact and interpretable architecture for structured QA.

The bAbI benchmark (Weston et al., 2015) provides a controlled testbed: 20 tasks of increasing difficulty, each probing a specific reasoning skill (counting, coreference, path-finding, etc.), with a vocabulary of roughly 145 words so that the challenge is purely in the reasoning, not in language understanding. The original paper reports a mean test error of 6.6% across tasks using the 10k training set; Tasks 16 (basic induction), 17 (positional reasoning), and 19 (path finding) remain the hardest, with errors of 0%, 40.7%, and 66.5% respectively.

Research questions. This project addresses two questions:

1. Can we reproduce the MemN2N results on bAbI, and where do gaps with the paper arise?

2. Does supervising the model's attention with supporting-fact labels help on hard tasks, and if so, why only on some of them?

The second question matters because it probes *why* a task is hard. If a model fails because it looks at the wrong sentences, directing its attention should help. If it fails because the sentence representations are inadequate or the architecture lacks capacity, better attention does not fix anything.

2 Related Work

Memory-augmented networks. Memory Networks (Weston et al., 2014) were the first architecture to pair a neural module with an explicit, addressable memory store. They showed strong results on bAbI but required strong supervision: the model was told during training which memory entries were relevant at each step. MemN2N (Sukhbaatar et al., 2015) relaxed this by making the memory reads fully differentiable, removing the need for intermediate supervision and making the model trainable end-to-end. A parallel line of work, Neural Turing Machines (Graves et al., 2014), uses a more general differentiable memory with both read and write heads, enabling algorithmic tasks that go beyond QA, at the cost of significantly higher complexity. In comparison, MemN2N's read-only design is simpler and more suitable for reproducibility studies on structured QA.

Attention mechanisms. The soft attention over memory in MemN2N is closely related to the alignment mechanism introduced for neural machine translation by Bahdanau et al. (2015). In both cases, a query vector computes a compatibility score with each element of a set (memory slots or encoder hidden states), and the result is a weighted sum used downstream. MemN2N extends this with multiple attention rounds (hops), allowing the model to iteratively refine which part of the memory it focuses on, similar in spirit to the iterative passage reading in later comprehension models.

Iterative memory networks. Dynamic Memory Networks (Kumar et al., 2016) are a more expressive successor to MemN2N: they encode sentences with an RNN rather than a bag-of-words, and use a gating mechanism at each memory pass. This gives stronger performance on several bAbI tasks, particularly those sensitive to word order, but at higher computational cost. Our work deliberately stays with the simpler MemN2N to study the specific effect of attention supervision without confounding factors from the encoder.

Semi-supervised learning in NLP. Using partially labeled data to guide a model's internal representations is a common idea in NLP. Self-training, consistency regularization, and auxiliary losses have all been applied to sequence labeling and reading comprehension. The closest work to ours is the original Memory Networks paper, where supporting-fact labels were used at training time to guide memory reads via a max-margin loss. We revisit this idea in a softer form: a KL-divergence auxiliary loss applied only to labeled examples, to measure how much guidance is needed and for which tasks.

3 Method

3.1 MemN2N Baseline

We re-implement the MemN2N architecture from Sukhbaatar et al. (2015) in PyTorch with $K = 3$ hops and adjacent weight tying. The overall architecture is shown in Figure 1.

Memory reads and hops. At each hop k, the model encodes the story sentences into input representations $\{m_i^k\}$ and output representations $\{c_i^k\}$ using embedding matrices A^k and C^k. Attention weights and the updated query are computed as:

$$p_i^k = \mathrm{Softmax}\left(u^{k\top} m_i^k\right), \qquad o^k = \sum_i p_i^k \, c_i^k, \qquad u^{k+1} = u^k + o^k.$$

The final answer prediction is $\hat{a} = \mathrm{Softmax}(W\, u^{K+1})$, where $W^\top = C^K$ under adjacent tying.

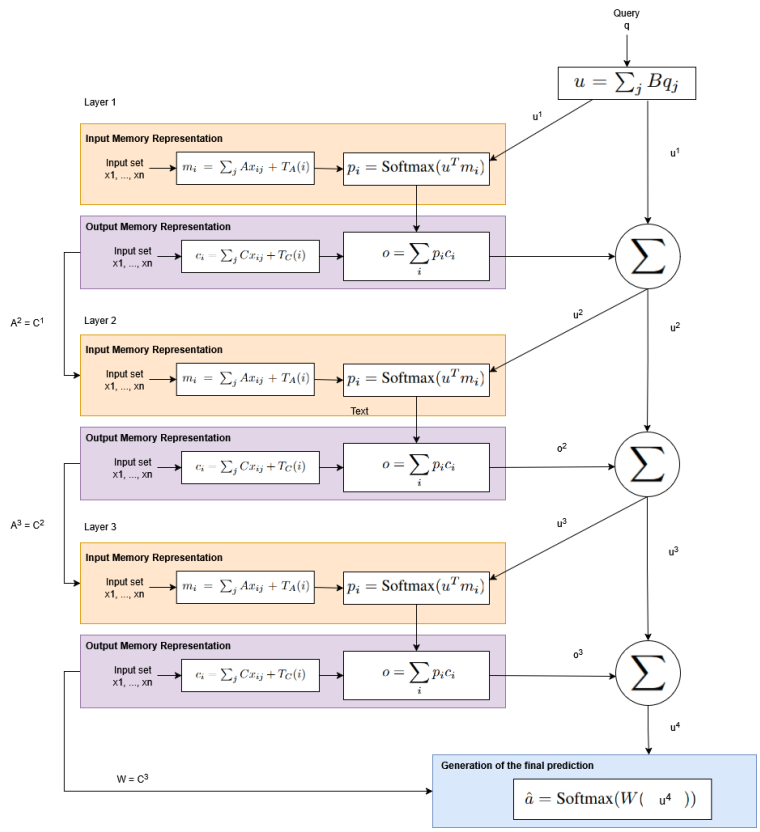

Figure 1: MemN2N with $K = 3$ hops and adjacent weight tying. The query u^1 is refined through three memory reads before the final linear prediction. $A^{k+1} = C^k$ under adjacent tying.

Encodings. Sentence vectors are formed with *position encoding*: each word embedding is scaled by a position-dependent coefficient before summing, so that word order affects the memory representation. We add *temporal encodings* T_A, T_C — one embedding per memory slot — to give the model a sense of recency.

Linear Start (LS). We train without the attention softmax for the first 20 epochs. Without this schedule, the softmax produces near-uniform attention at initialization and the gradient signal is too weak for the embeddings to separate.

Random Noise. Following the paper, we randomly zero out 10% of memory slots at each step to prevent overfitting to absolute temporal positions.

3.2 LSTM Baseline

We also train a bidirectional LSTM (Hochreiter & Schmidhuber, 1997) as a reference. Sentence vectors (bag-of-words embeddings) are fed sequentially; the final hidden state concatenated with the query embedding is passed through a linear classifier. This matches the LSTM column in Table 1 of the original paper and measures how much explicit memory helps over sequential processing.

3.3 Semi-Supervised Extension

The bAbI data includes supporting-fact indices in the raw files: each question line lists the sentence numbers used to answer it. Standard MemN2N training ignores these. Our extension uses them as weak supervision for the hop-1 attention.

For each labeled training example we construct a binary mask $M \in \{0,1\}^{N_{\text{mem}}}$ with $M_i = 1$ if sentence i is a supporting fact, then normalize it to get a target distribution $\hat{M}$. The training objective becomes:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{QA}}}_{\text{all examples}} + \lambda \underbrace{\text{KL}\Big(\hat{M} \parallel p^1\Big)}_{\text{labeled examples only}},$$

where p^1 is the hop-1 attention distribution and $\lambda = 0.5$. Both losses use `reduction='sum'`, so λ controls the KL share consistently regardless of batch size. At 50% labeled the KL term contributes roughly 25% of the total loss. At 0% labeled no example satisfies the label condition, so the objective reduces exactly to the unsupervised baseline. The KL term is only activated after the Linear Start phase ends (epoch > 20), since raw LS scores are not a valid probability distribution.

4 Experimental Setup

Dataset. We use the bAbI dataset (Weston et al., 2015), specifically the 10k training set (en-10k). Each of the 20 tasks contains 10,000 training examples and 1,000 test examples, with a vocabulary of approximately 145 unique words. Stories vary considerably in length: Task 1 averages about 5 sentences, while Task 3 can reach 320 (see Figure 5 in the Appendix for a full dataset overview).

Evaluation metric. We report **test error rate** (percentage of test questions answered incorrectly), following the original paper. For the semi-supervised experiments each condition is run 3 times with different random seeds and results are averaged. The paper uses 10 restarts; we use 3 for computational reasons.

Hyperparameters. Table 1 lists the parameters shared across experiments.

Table 1: Hyperparameters for all models.

Parameter	MemN2N / SS-MemN2N	LSTM
Embedding dim	20	64 (hidden)
Number of hops	3	—
Optimizer	SGD	Adam
Initial LR	0.005	0.001
LR schedule	halved every 25 epochs	halved every 25 epochs
Gradient clipping	40	40
Batch size	32	32
Epochs	100	100
Linear Start	20 epochs	—
Random Noise	10% slots zeroed	—
Memory size cap	50 slots (150 for Task 3)	—

Semi-supervised experiment. We evaluate the semi-supervised model on Tasks 16, 17, and 19 — chosen because they represent three qualitatively different failure modes of the unsupervised baseline (51.2%, 41.6%, and 58.6% error respectively). We test four labeling ratios: 0%, 10%, 25%, and 50% of training examples, with auxiliary weight $\lambda = 0.5$ and KL supervision applied to hop-1 attention only.

5 Results and Discussion

5.1 Baseline: MemN2N vs. LSTM vs. Paper

Table 2 shows per-task results across all 20 bAbI tasks (Figure 6 in the Appendix gives a visual comparison). Our MemN2N achieves a mean test error of 10.8%, compared to 6.6% in the paper. The LSTM reaches 37.7%, in line with the paper's 39.5%. The broad ordering is well reproduced: MemN2N is substantially better than LSTM on tasks requiring multi-hop chaining (Tasks 2, 3, 6,

Table 2: Test error (%) on all 20 bAbI tasks. **Ours (M)** is our MemN2N re-implementation; **Ours (L)** is our LSTM baseline. **Paper M / Paper L** are from Sukhbaatar et al. (2015). † marks tasks where our error exceeds the paper by more than 5 points.

Task	Name	Ours (M)	Ours (L)	Paper M	Paper L
1	single-supporting-fact	0.0	47.9	0.0	0.0
2	two-supporting-facts	1.7	60.9	0.3	81.9
3	three-supporting-facts	16.5†	56.4	9.3	83.1
4	two-arg-relations	0.0	32.1	0.0	36.6
5	three-arg-relations	0.6	18.4	0.8	1.2
6	yes-no-questions	0.2	50.8	0.1	51.8
7	counting	7.7	19.3	3.2	24.9
8	lists-sets	7.1†	21.6	0.9	22.8
9	simple-negation	0.2	38.0	0.3	20.2
10	indefinite-knowledge	0.3	52.6	0.0	30.1
11	basic-coreference	0.0	29.1	0.0	10.3
12	conjunction	0.0	24.3	0.0	23.4
13	compound-coreference	0.0	5.6	0.0	6.1
14	time-reasoning	1.4	50.3	0.0	81.0
15	basic-deduction	0.0	41.7	0.0	78.7
16	basic-induction	51.2†	51.2	0.0	50.1
17	positional-reasoning	41.6	54.1	40.7	50.1
18	size-reasoning	9.1	8.9	9.2	45.8
19	path-finding	58.6	89.3	66.5	90.3
20	agents-motivations	19.1†	1.8	0.0	2.1
Mean		**10.8**	**37.7**	**6.6**	**39.5**

9, 14) and at most tasks overall. The gap with the paper comes from four tasks: Task 3 (16.5% vs. 9.3%), Task 8 (7.1% vs. 0.9%), Task 16 (51.2% vs. 0.0%), and Task 20 (19.1% vs. 0.0%). These are exactly the tasks that benefit most from joint multi-task training, which the paper uses but we do not.

The Linear Start phase is critical. Figure 2 shows that once the softmax is reintroduced at epoch 20, training accuracy on Task 1 converges to 100% and loss continues to decrease for the remaining 80 epochs. The LSTM never converges below 58% training accuracy, confirming that sequential processing alone cannot handle even simple two-step fact retrieval.

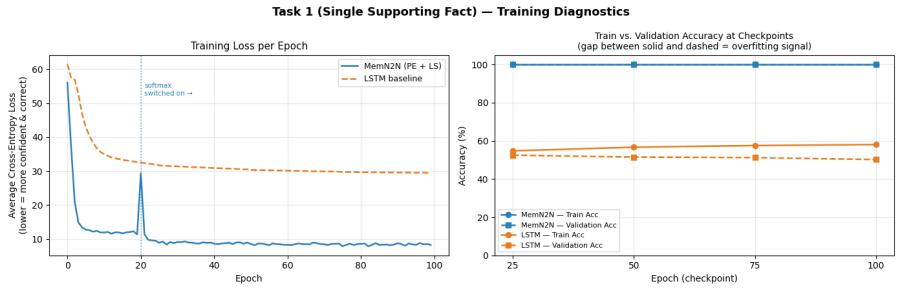

Figure 2: Training curves for Task 1. Left: cross-entropy loss. Right: accuracy. The shaded band marks the Linear Start phase (epochs 1–20); accuracy jumps to 100% immediately after softmax is restored.

Why does Task 16 fail? Task 16 (basic induction) asks the model to identify a rule pattern (e.g. "Wolves are afraid of cats. Cats are afraid of mice." → "What are wolves afraid of?"). The paper achieves 0% with MemN2N PE+LS on 10k training; our single-run baseline gives 51.2% — essentially random guessing. The failure is not due to insufficient reasoning capacity but to the model never locating the rule sentence to begin with.

Why does Task 19 fail? Task 19 (path finding) requires chaining 5+ locations together, but a 3-hop model can at most chain 3 steps. The paper's 66.5% error confirms the architecture is under-powered for this task regardless of training.

5.2 Semi-Supervised Results

Table 3: Semi-Supervised MemN2N test error (%) on Tasks 16, 17, and 19. **MemN2N** is the unsupervised baseline. **0%–50%** are mean errors over 3 runs at each labeling ratio. **Paper** is from Sukhbaatar et al. (2015).

Task	Name	MemN2N	0%	10%	25%	50%	Paper
16	basic-induction	51.2	35.3	34.5	**0.4**	**0.4**	0.0
17	positional-reasoning	41.6	38.8	50.1	52.6	52.7	40.7
19	path-finding	58.6	62.2	63.3	66.7	66.1	66.5

Table 3 and Figure 3 show three distinct patterns across the three tasks.

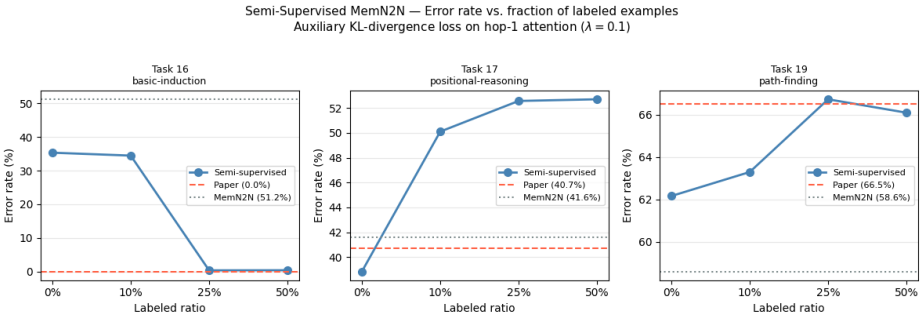

Figure 3: Semi-supervised training curves for Tasks 16, 17, and 19 at each labeling ratio. Task 16 converges to near-zero error at 25% labeled; Tasks 17 and 19 show no benefit, reflecting representational and architectural bottlenecks.

Task 16 — supervision nearly solves it. At 25% labeled, error drops from 51.2% to 0.4%, nearly matching the paper's 0.0%. The improvement is sharp and concentrated between 10% and 25% labeled, suggesting a critical mass of examples is needed before the model reliably learns to locate the rule sentence. The attention visualization (Figure 4) confirms this: the supervised model concentrates its hop-1 attention on the induction rule sentence, while the unsupervised model spreads it nearly uniformly.

Task 17 — supervision hurts slightly. Error rises from 41.6% to 52.7% at 50% labeled. Task 17 requires reasoning about spatial relations ("X is left of Y"). A bag-of-words encoder treats "X is left of Y" and "Y is left of X" identically, so even perfect attention cannot extract the spatial relation. Forcing the model to focus on the correct sentence via KL loss interferes with whatever approximate strategy it had found unsupervised — confirming that when the representation is the bottleneck, better attention makes things worse.

Task 19 — supervision is irrelevant. Error increases monotonically to 66.7% at 25% labeled, essentially matching the paper's 66.5% at that point. Path finding requires chaining 5+ steps but a 3-hop model cannot do this. Directing attention to supporting facts does not add hops and cannot compensate for the architectural capacity gap.

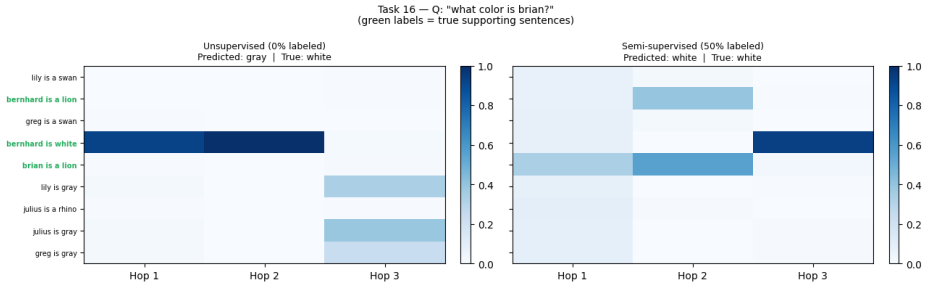

Figure 4: Hop-by-hop attention heatmaps for a Task 16 test example. Left: unsupervised (diffuse attention). Right: 50% labeled (sharply focused on the induction rule sentence at hop 1). Supporting facts are highlighted in green.

What the 0% labeled condition reveals. At 0% labeled (no KL term at all), errors are 35.3%, 38.8%, and 62.2% for Tasks 16, 17, and 19 respectively, vs. the single-run baseline of 51.2%, 41.6%, and 58.6%. The differences reflect run-to-run variance rather than any algorithmic difference: Task 16 in particular has high variance between restarts, and the 3-run average is simply more stable.

6 Conclusion

We re-implemented MemN2N in PyTorch and reproduced the key results of Sukhbaatar et al. (2015) on bAbI, achieving a mean test error of 10.8% against the paper's 6.6%. The remaining gap is concentrated on tasks (3, 8, 16, 20) that benefit from joint multi-task training, which the paper uses but we do not. Our LSTM baseline closely matches the paper's LSTM results, confirming that the benefit of explicit memory is real.

The semi-supervised extension answers the diagnostic question we posed: attention supervision helps only when the failure is attentional. For Task 16, 25% labeled examples nearly closes the gap to the paper's 0.0%. For Tasks 17 and 19, supervision makes things slightly worse because the bottleneck is representational (bag-of-words cannot encode spatial relations) or architectural (3 hops cannot chain 5 steps).

Limitations. This study trains each task independently rather than jointly. We use embedding dimension 20, which may limit capacity on harder tasks. The 3-run average is noisier than the paper's 10-run average, particularly for high-variance tasks like Task 16.

Future directions. Three extensions seem most promising: (1) joint multi-task training to improve Tasks 3, 8, and 20; (2) replacing the bag-of-words encoder with a position-aware encoder (e.g. the ReLU variant from the paper) to give Task 17 a chance; and (3) increasing the number of hops or using an adaptive hop architecture for path-finding tasks.

References

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. volume 9, pp. 1735–1780, 1997.

Ankit Kumar, Ozan Irsoy, Peter Ondruska, Mohit Iyyer, James Bradbury, Ishaan Gulrajani, Victor Zhong, Romain Paulus, and Richard Socher. Ask me anything: Dynamic memory networks for natural language processing. In *International Conference on Machine Learning*, pp. 1378–1387, 2016.

Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. *Advances in neural information processing systems*, 28, 2015.

Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. *arXiv preprint arXiv:1410.3916*, 2014.

Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M Rush, Bart Van Merriënboer, Armand Joulin, and Tomas Mikolov. Towards ai-complete question answering: A set of prerequisite toy tasks. *arXiv preprint arXiv:1502.05698*, 2015.

A Additional Figures

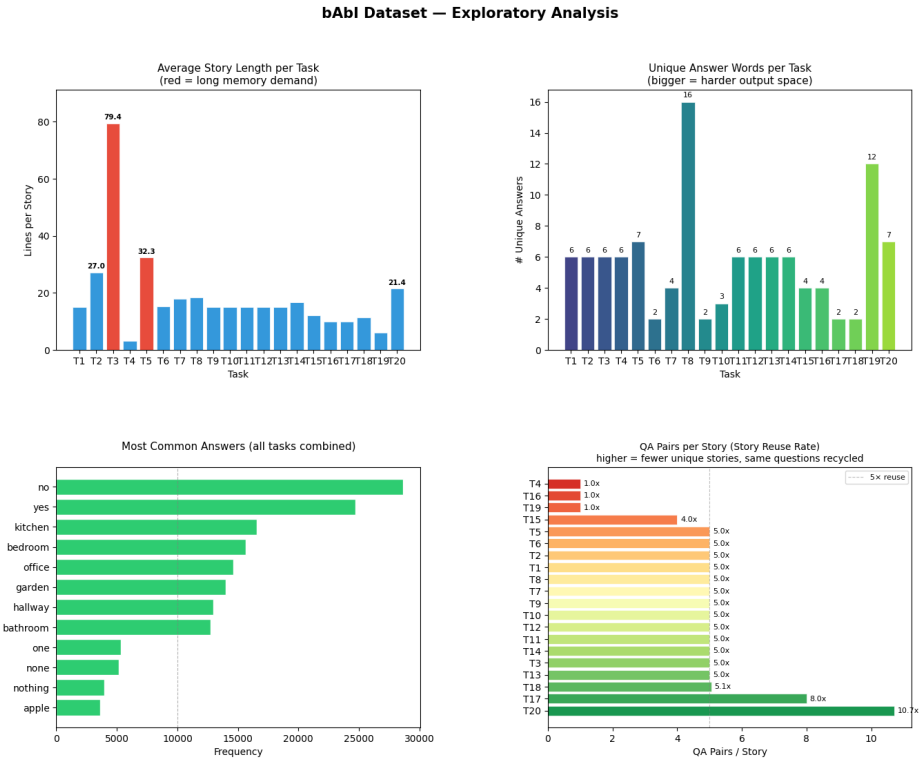

Figure 5: Exploratory analysis of the bAbI dataset. Top-left: average story length per task (Task 3 is the longest at up to 320 sentences). Top-right: answer frequency distribution, dominated by location words. Bottom panels: vocabulary coverage and question-length distribution.

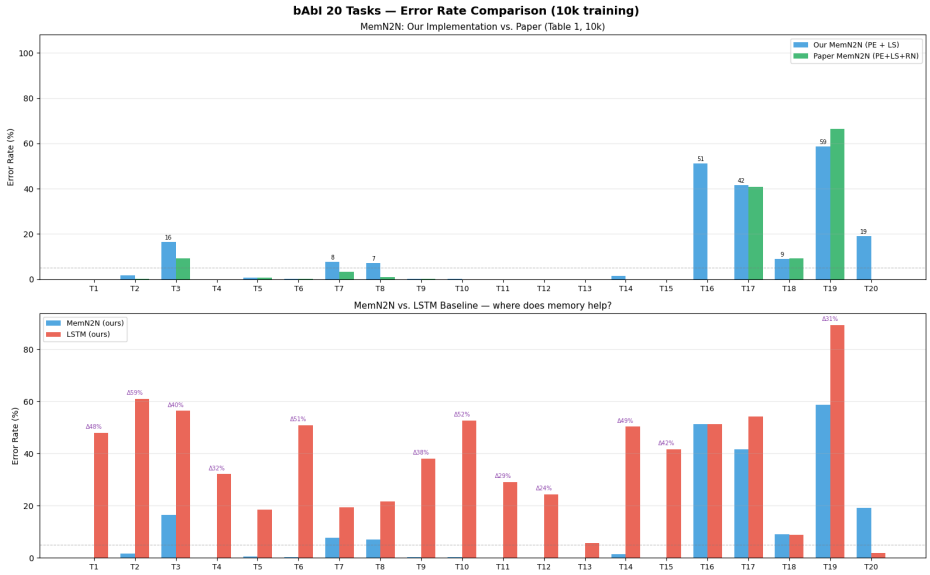

Figure 6: Per-task error comparison between our MemN2N (blue), our LSTM (orange), and the paper's MemN2N (dashed grey). Our model tracks the paper closely on most tasks; the main gaps are Tasks 3, 8, 16, and 20.